Assessing transformer scalability across many days of neural decoding

ROB 590 - Report

Omer Benharush
Mentor: Joey Costello
Supervisor: Prof. Cynthia Chestek
CNPL - University of Michigan

Introduction

Recent successes of large language models have shown the potential of transformer architectures [1]. In particular, these advances have shown that scaling up the size of the training set can drastically improve model performance. Unfortunately, decoding from neurons is more complex than decoding from language. Unlike words in language, neurons shift over time so that subsequent days cannot be aligned [2]. To overcome this, neural transformer models utilize various tokenization techniques to group together like neurons while maintaining temporal and spatial information [3,4]. Further, these models incorporate fine tuning layers for the specific decoding task. With these additions, large neural transformer models have shown promising results.

As an initial step towards replicating these complex models, this report investigates whether stitching together training data from many different sessions improves single day performance of a transformer decoder when compared training on a single day.

Methods

The first step in implementing this project was processing the large dataset Hisham created into a PyTorch data loader. To accomplish this, I wrote a function that iterated over a specified number of days, extracting spiking band power and finger kinematics for each day. Then each day's data was split between training (80%) validation (10%) and testing (10%). The kinematics were then normalized and the day's data was appended to the larger train, validation and test set.

I used the standard TFM Decoder from the lab's PyBMI library as the core decoder model. To account for training across days, I used the multi-day wrapper Joey previously developed with a day specific input layer with size of 96 to match the channel count. This wrapper creates a unique linear read-in layer for each day. I used the default dropout percentage of 0.1 for the model. For training the model I used the train_model() function from Training_Utils in PyBMI with the multi-day flag on. I used the Adam optimizer, with a weight decay of 0.002, along with the ReduceLROnPlateau scheduler with factor of 0.5 and patience of 8.

To decide on learning rate, encoder hidden size and number of encoder hidden layers for the model, I performed a parameter sweep where I trained one model for each set of hyper-parameters on ten days of data for 10k iterations without a scheduler. The results of the sweep can be seen below in Table 1. I noticed a significant difference only for the learning rate where the default of 2e-4 was best. For the encoder hidden size and number of hidden layers, I went with the default values of 1000 and 4

respectively as there was no clear best choice from the parameter sweep data.

Learning Rate	2.0e-06	2e-05	2e-04	2e-03
Average Final Training Loss	0.90951	0.69210	0.58809	0.67698
Encoder Hidden Size	256	512	1000	5000
Average Final Training Loss	0.78682	0.77811	0.77020	0.75488
Encoder Hidden Layers	1	2	3	
Average Final Training Loss	0.79070	0.76514	0.76167	

Table 1. Parameter sweep across learning rates, encoder hidden size and encoder hidden layers. The performance metric used is average MSE for the last iteration of training. Lower loss values indicate better performance.

Using these hyper-parameters, I trained models on four datasets: the first 10 days, the first 50 days, the first 100 days, and the first 200 days. These models were trained with the scheduler and a maximum iteration count of 100k.

For the single day models, I trained only one model per day for each of the first 200 days using the default learning rate, encoder size, number of layers, and scheduler.

To assess model performance I tested each of the multi-day models and the corresponding single day model on the test set for the applicable subset of the first 200 days of data. The correlation coefficient between the predicted kinematics and ground truth was recorded for each model for each day and can be seen in Figure 2.

Results

As can be seen in Figure 1, it appears that training across multiple days has the potential to augment decoder performance. The box plot in Figure 1 shows each decoder's performance, relative to the applicable single day model, for the first ten days of the test set.

It is notable that the mean of the first model is positive, showing improvement over the single day model. It is also interesting to note, that can be seen in Figure 2, that the correlations of the multi-day models seem to match themselves even when diverging from the single day model. This, along with the worse performance of larger models in Figure 1, could be due to the multi-day models working to minimize the loss across all training days, whereas the single day model only minimizes the loss for the single day.

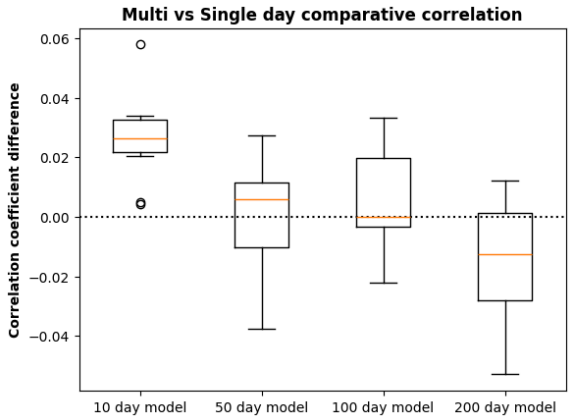

Figure 1. A box plot comparing the 4 multi-day models over their first ten days of data to the corresponding single day model is shown with correlation coefficient as the metric.

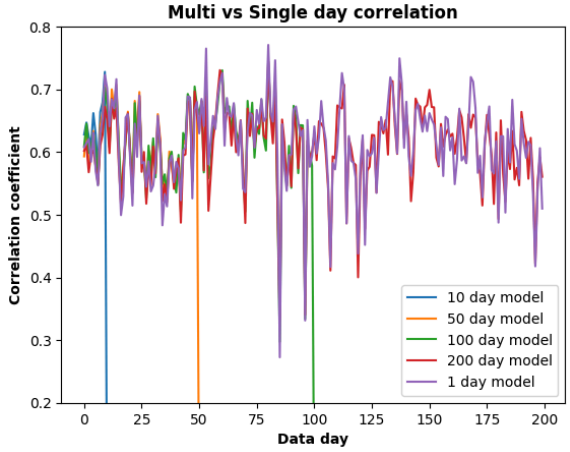

Figure 2. Correlation coefficient for each model across all of its testing days. Note that models drop to

the bottom of the screen when the day number exceeds their training days, as each model learns a specific input layer for each individual day.

Discussion

That note brings up a central issue with neural decoders: to what degree is past neural data helpful in decoding current neural activity? For the ten day model, the median correlation improved by 0.02 as compared to the one day models. One tailed T-test shows that the multi-day model is an improvement with p-value 0.106. While this p-value is not low enough to be significant, it shows potential for this area of work. Additional techniques such as fine tuning the model on training data from the test day, so as to more heavily bias the model towards the specific neural data of the test day, and a more complex encoder, which can encode the data in a richer space and find finer relationships between neural features, can help balance out some potential negative side effects of increasing the days in the training set.

The intended extension for the project is to replicate a more complex encoder and transformer architecture, such as the one used in the Poyo model. Much of the code from this project would generalize to implement such a model. In the coming weeks, the author's of the Poyo paper are planning on publishing a tutorial for how to implement their architecture along with examples. Further, Joey and I have been in touch with the first author of the Poyo paper, and he has indicated that he's willing to answer any questions we might have after the tutorial is released.

Appendix

Another thing I looked at during this project was whether scaling up the number of encoder parameters, i.e. increasing the hidden size and number of layers in the encoder, would improve performance as data size grew. As can be seen in Table 2 below, this does not seem to be the case.

Hidden layers \ Hidden size	256	512	1000	5000	
2	0.61277	0.62159	0.62515	0.61295	
3	0.62575	0.62292	0.62496	0.61864	50 day model
4	0.62128	0.62396	0.62342	0.61785	
2	0.60638	0.60631	0.60299	0.60507	
3	0.60936	0.61129	0.60029	0.59612	100 day model
4	0.60374	0.59687	0.60719	0.59598	
2	0.61297	0.60486	0.60927	0.59849	
3	0.61135	0.60655	0.60912	0.60254	200 day model
4	0.60834	0.60644	0.60980	0.60618	

Table 2. Average correlation coefficient for each model for each hyper-parameter set.

While training the TFM Decoder from PyBMI, a user warning popped up. Here they are and this is what they mean:

- UserWarning: To copy construct from a tensor..

In PyBMI there is an efficient function used to copy a tensor. I do not believe it practically affects anything for the lab.

- UserWarning: 1Torch was not compiled with flash attention.

PyTorch has multiple different ways to compute attention blocks and will by default choose whichever one it thinks will be the most efficient during runtime. I found that the only way to avoid cuda crashing when training a larger transformer model was by explicitly setting flash, one of those methods, to false, and math_sdp, the c++ method, to true.

Great lakes is a fantastic underutilized resource that this lab has access to. I would

greatly recommend trying it out. Here's a quick tutorial on how to get it running. First you need access to the lab's account and then you need to request a user profile for yourself. Then you can either ssh straight in or go through the GUI. I typically go through the GUI and select Jupiter lab to start a session. When you start the session you can select what resources to request. Once you request a resource for an amount of time, that gets charged to the lab's account even if it goes unused. In terms of speed, I've found the GPU partition on great lakes is about 15% faster than brainiac. That being said, trying to use multiple GPUs without properly parallelized code will underperform a single gpu. When starting up, you need to set up your python env just like for any other computer. These are some good GitHubs to check out.

As no one else was trying to use brainiac when I was training, and as my code was insufficiently parallelized, I did most of the training for this project on brainiac. Table 3 below has approximate training times for the schedule mentioned in the methods section using brainiac.

Number of training days	1	10	50	100	200
Approximate training time	1-3 min	10-20 min	30-45 min	45-90 min	150-360 min

Table 3. Approximate training time for each data size.

References

[1] Yiheng Liu, Tianle Han, Siyuan Ma, Jiayue Zhang, Yuanyuan Yang, Jiaming Tian, Hao He, Antong Li, Mengshen He, Zhengliang Liu, et al. 2023. Summary of ChatGPT/GPT-4 research and perspective towards the future of large language models. arXiv preprint arXiv:2304.01852 (2023).

[2] C. Pandarinath, K. C. Ames, A. A. Russo, A. Farshchian, L. E. Miller, E. L. Dyer, and J. C. Kao, "Latent factors and dynamics in motor cortex and their application to brain–machine interfaces," Journal of Neuroscience, vol. 38, no. 44, pp. 9390–9401, 2018.

[3] M. Azabou et al., A unified, scalable framework for neural population decoding. *Adv. Neural Inf. Process. Syst.*36, 44937 (2024).

[4] Joel Ye, Jennifer Collinger, Leila Wehbe, and Robert Gaunt. Neural data transformer 2: multicontext pretraining for neural spiking activity. bioRxiv, pages 2023–09, 2023.